

En synopse over programmering B
eksamensopgave baseret på DND

Eksamensopgave

Initiativetracker

Skole: Slotshaven gymnasium

Fag: Programmering B

Lærer: Mikkel Kirkeby Mosthaf

Dato for aflevering: 27/03/26

Applikation af: Jeppe Skovgaard
Andersen og Stefan Dan Jensen

Repo-link:
<https://github.com/DysL3xx/GitForEksamenOpgave/tree/main>

Synopse af: Stefan Dan Jensen

Endeligt produkt:

D&D Initiative Tracker

Add Character

Name:

Initiative:

Type:

Image:

Select Image

Add Character

← Previous Turn

Turn 1/4: EnemyBoss's turn

Next Turn →

Turn Order (Highest Initiative First)

EnemyBoss (Initiative: 20) - Enemy

Hero#2 (Initiative: 10) - Hero

Ally#1 (Initiative: 7) - Ally

Hero#1 (Initiative: 5) - Hero

Select a character to view image

Remove Selected

Clear All

Indholdsfortegnelse

Opgaveformulering.....	3
Projektbeskrivelse.....	4
Kravspecifikation.....	4
Gruppefordelingen	4
brug af AI.....	5
Funktionsbeskrivelse	5
Dokumentation	8
Kode – eksempler	10
Colorscheme:.....	10
Constructor funktion:.....	11
alle sektioner i applikationen:.....	12
kode – generelt	12
Dokumentation af AI-brug.....	12
Test.....	13
Testresultater	13
Konklusion	13
Bibliografi	14
Bilag	15

Opgaveformulering

Tema 2. Python-applikation med eget tema

I skal udvikle en python-applikation som demonstrerer brug struktureret brug af trelagsmodellen. Det vil sige at løsningen skal indeholde et data-, et logik- og et præsentationslag. Datagrundlaget og præsentationslaget er valgfrit - I bestemmer altså selv hvad løsningen handler om og hvordan den præsenteres for brugeren.

Projektbeskrivelse

Vores produkt kommer til at bestå af en "initiative tracker" til bordrollespillet "Dungeons and Dragons", det består af en applikation der kommer til at holde styr på rækkefølgen hvor ved at de forskellige spillere og/eller fjender kommer til at agere i spillet. Det kommer til at fungere som et administrativt værktøj for "game masteren"(GM) så han kan holde styr på spillets løb. Det er ofte at spillere holder styr på det ved at skrive navne ned på et papir i den rækkefølge de agere på, men fejl sker ofte og det bliver især svært når fjender ikke har rigtige navne men i stedet bare har numre(når der er flere af samme slags), hvilket er hvorfor vores applikation er nødvendigt.

Kravspecifikation

Vi kommer hovedsageligt til at fokusere på 5 krav for at sikre os at vores produkt kommer til at være både pænt, lovligt og brugbart, de krav er:

Forståelig for nybegyndere: den skal helst være simpel da det er et værktøj der har et hovedmål for dets brug og alt andet bare kommer til at være side additioner til det endelige produkt.

Let forståelig opbygning af knapper: den skal interaktivt være stærk da det er et værktøj der helst skal være så intuitivt og effektivt i de processer brugen har hentet applikationen for.

Farver der passer til vores tema: den skal helst have et design som er nemt at forstå og behageligt at kigge på, og som passer ind i den applikation som vi vil lave.

Opholdning af ophavsret: da det er en applikation baseret på et eksisterende spil, hvor rettighederne allerede er eget og de artistiske valg vi tager, kommer til at krænke ejerne af Dungeons and dragons' ophavsret.

Intet delay: vores applikation skal som sagt være simpel, det er derfor vigtigt at denne simplicitet gør at den funktion den fokuserer på, derfor kommer til at virke uden problemer eller forsinkelser.

Krav	Vurdering
Forståelig for nybegyndere	Nice-to-have
Let forståelig opbygning af knapper	Need-to-have
Farver der passer til vores tema	Nice-to-have
Opholdning af ophavsret	Need-to-have
Intet delay	Nice-to-have

Gruppefordelingen

Her kommer vi til at skrive om de fokusområder som vi hver især har inden for produktudviklingen.

Stefan kommer primært til at være vores GUI person, så han kommer til at stå for hvordan selve brugerne ser, oplever og interagerer med vores applikation.

Jeppe kommer til at være vores database person, så han kommer primært til at stå for vores database, hvordan den interagerer med brugerne og hvordan den håndterer information som den får fra brugerne

Logik laget kommer til at være fordelt ligeligt mellem de to gruppemedlemmer og tilbygninger til det laves, når brugen opstår indenfor de andre lag, for eksempel hvis databasen skal bruge kode på logiklaget for at spille sammen med GUI'en

brug af AI

vores AI brug kommer til at bestå af hovedsageligt af hjælp til programmering af især styling og database da dette ofte sker i sprog som vi ikke har fået fordybende læring i. vi kommer til at bruge AI som et værktøj til at komme med forslag og mulige ændringer frem for at den bare laver programmet for os, dette gøres for at maksimere både læring og personligt input og præg på det endelige produkt. Vi kommer til at gøre dette ved hjælp af instruktionsfiler skrevet inde i workspacet så AI'en følger de retningslinjer vi giver den, hvilket blandt andet kommer til at indeholde dokumentation af AI-brug, som dokumenterer både prompt, svar og dato af de forskellige interaktioner vi har med AI.

Funktionsbeskrivelse

De funktioner som vores program består af, er. En funktion til kreation af entiteter, som tillader en at navngive entiteten og bestemme dens initiativ, samt en funktion der lader en differentiere imellem entiteter ved at tilskrive dem enten statussen, Hero, Ally and Enemy. Derudover er der også en displayliste til disse entiteter som brugeren laver. Alle disse funktioner fungerer ved kald og samspil med vores database til lagring af dataet over entiteter.

De funktioner der står for navigation er 4 forskellige knapper der styrer hvad listen viser, de 4 knapper er "next turn", "previous turn", "remove selected" og "clear all". "turn" knapperne står for hvilken karakter der er på toppen af listen også kaldet den, hvis tur det er. "Remove selected" lader en trykke på en af karaktererne og sige at den specifikt skal slettes og "clear all" står for bare at slette alle de karakterer der er på listen.

The screenshot shows a web application titled "Initiative Tracker". It features a dark-themed interface with yellow buttons. The top section, titled "Add Character", contains three input fields: "Name:", "Initiative:", and "Type:". The "Type:" field is a dropdown menu currently showing "hero". Below these fields are three buttons: "Add Character", "next turn", and "previous turn". The bottom section, titled "Turn Order", contains a large, empty rectangular area with a scroll bar on the right side. At the bottom of the application are two buttons: "Remove Selected" and "Clear All".

Figur 1, fremvisning af 3. iteration af brugergrænsefladen

Man kan desuden se her at vi gør brug af både, input felter, dropdown-menuer og scrollbarer, for at optimerer vores brugergrænseflade og brugerens oplevelse applikationen. De forskellige funktioner som opgør vores applikation er desuden opdelt i mindre sektioner for at give brugeren bedre overblik. Og farverne stemmer overens med "wizards of the coast"(WotC), som er ejerne af ophavsretten på dnd, og sørger for at det ligner lignende redskaber som brugeren vil bruge. Selve applikationen kommer til at fylde 500*600 men tillader brugeren af forstørre eller formindske vinduet.

Initiative Tracker

Add Character

Name:

Initiative:

Type:

Turn Order

Featherine Fullbring (Initiative: 25) - Hero
Zombie#3 (Initiative: 19) - Enemy
Azith Grimm (Initiative: 15) - Hero
Zombie#2 (Initiative: 9) - Enemy
Zombie#1 (Initiative: 7) - Enemy
Michael Michael Michaelson (Initiative: 5) - Hero
soldier#1 (Initiative: 1) - Ally

Figur 2, fremvisning af 3. iteration i brug

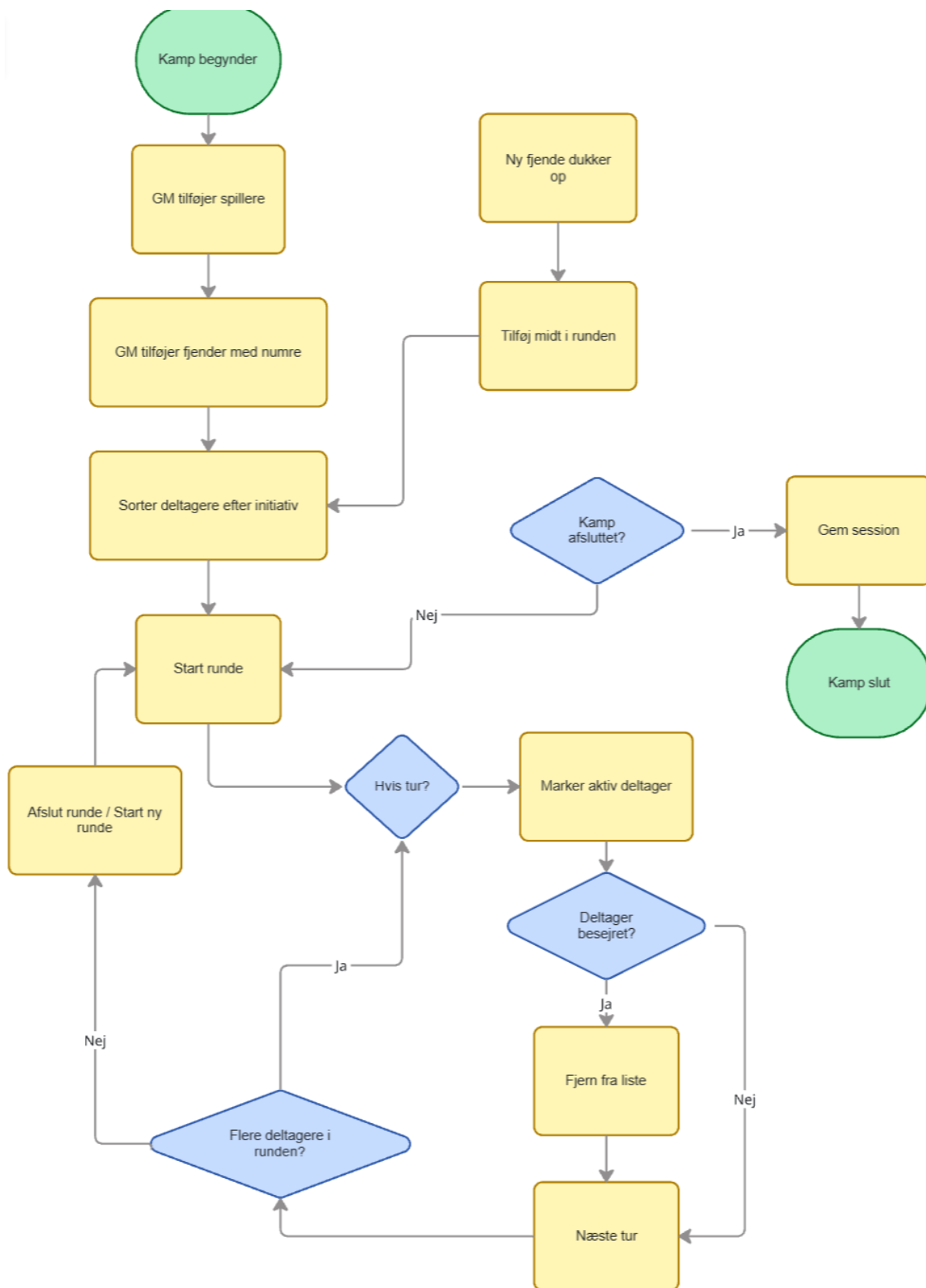
Dokumentation

Vi har valgt at opbygge vores program med en struktur hvor de forskellige dele er opdelt efter deres funktion, så det er nemmere at skabe overblik og arbejde videre med. Programmet består overordnet af en database del, en brugergrænseflade og nogle logikfiler. Denne opdeling gør at vi kan holde styr på hvor de forskellige funktioner ligger, og hvordan de arbejder sammen i systemet.

Database delen ligger i mappen DB og består af både selve databasen og den Python kode der håndterer lagring og hentning af data. Det er her alle informationer om entiteter bliver gemt, og hvor funktionerne der opretter, henter og sletter data bliver udført. Brugergrænsefladen ligger i mappen GUI og består af en HTML fil til struktur og en Python fil som håndterer brugerens input og sørger for at sende vores data videre til databasen. Derudover har vi en mappe med statiske filer som indeholder et billede der bliver brugt i programmet som logo.

Programmet starter i filen main.py som står for at samle de forskellige dele og få dem til at fungere sammen. Vi har også en separat fil til data og datastrukturer som bliver brugt i programmet. Samlet set fungerer programmet ved at brugeren interagerer med brugergrænsefladen, hvorefter Python koden sender information videre til databasen og returnerer et resultat som bliver vist for brugeren.

Derudover har vi også både en instruktion og log fil til ai brug, dette gøres for at overholde rammerne om Ai brug og for at overholde de realistiske rammer for min egen forståelse af faget og programmet.



Figur 3, Miro flowchart over brug af applikation

Kode – eksempler

Her kommer jeg til at gå igennem eksempler af den kode jeg hr brugt i GUI'en, og hvad de står for

Colorscheme:

```
# Farver
BG_COLOR = "#1b1b1b"          # Deep charcoal background
PARCHMENT = "#2a2a2a"         # Dark card base
ACCENT = "#d4af37"             # Metallic gold for buttons
TEXT_LIGHT = "#f5eec7"         # Warm light text
TEXT_DARK = "#d6cfa3"         # Secondary text color

# karaktertype farver
HERO_COLOR = "#289928"         # Light green
ALLY_COLOR = "#6495ED"         # Cornflower blue
ENEMY_COLOR = "#FF7F50"        # Coral
```

hexcode over de farver der bliver brugt i applikationen, dette gøres både for design og overblik. Disse bliver refereret til ofte igennem resten af koden.

Constructor funktion og selve vinduets klasse:

```
class MainWindow:
    """
    klasse for hovedvinduet i GUI'en
    """

    def __init__(self, root, tracker):
        """
        Initialiserer hovedvinduet og opretter alle GUI-elementer.
        Args:
            root (tk.Tk): The root Tkinter window
            tracker (InitiativeTracker): The character tracker
        """
        self.root = root
        self.root.title("D&D Initiative Tracker")
        self.root.geometry("600x700")

        # gem tracker
        self.tracker = tracker

        # opret GUI-elementerne
        self.setup_ui()

        # vis karaktererne i trackeren
        self.update_all()
```

constructor funktion til brug af applikationen. Den starter og laver grundlaget af vores applikation og bestemmer vigtige funktioner og variabler til senere brug i koden. Denne klasse er hvad der er grundstenene for hele vores applikation.

karakterlisten:

```
# laver listen
self.character_listbox = tk.Listbox(
    self.frame,
    height=14,
    font=("Arial", 11),
    bg=PARCHMENT,
    fg=TEXT_LIGHT,
    selectbackground=ACCENT,
    selectforeground=BG_COLOR,
    activestyle="none"
```

dette er sektionen der laver den liste som kommer til at vise alle de forskellige karakterer, og er selve hoveddelen af vores applikation i brug.

kode – generelt

med mit fokus i GUI delen af vores applikation har størstedelen af min programmering foregået i et enkelt dokument, dette dokument fokuserer på at konstruere vores applikation og opstille alle de knapper lister og inputfelter nødvendigt for effektivt at kunne bruge vores app. I dette dokument er der dog også en smule af logiklaget for lettere håndtering af de forskellige funktioner. Fejlhåndtering, og textbox til "confirmation" af, for eksempel, sletning af entiteter i applikationen.

Dokumentation af AI-brug

Igennem vores projekt har vi brugt AI både konstruktivt og undersøgende til bearbejdning af kode og fejlfinding, dette gøres ved fremfor at få AI til bare at lave vores kode for os, så beder vi den om at komme med eksempler på løsningsmetoder og kode strategier, såsom hvor det vil være bedst at bruge statemachine programmering eller OOP. For at dokumentere dette har vi lavet et "AI-guideline" dokument der tvinger AI agenten til at følge en række retningslinjer der gør så den arbejder på et niveau jeg kan både forstå og lære af og lader mig bruge AI som et værktøj fremfor en krykke. Iblondt disse retningslinjer var der også kravet om en "AI-log" hvor der er dokumenteret dag, prompt og resultat for hver interaktion med den. Begge disse dokumenter ligger i vores repository. Vi har dog også forholdt os meget kritisk over den hjælp den giver, da AI ikke er perfekt og kan misforstå vores hensigter og spørgsmål. Vi brugte dog AI til at skrive vores testdokument fra pytest, da vores undervisning og forståelse for unittests ikke var tilstrækkelig til ordentligt at teste vores projekt.

Test

Den test jeg vil lave, er en "unittest", den er i meget simple termer, en måde at tjekke at dine funktioner outputter det rigtige resultat. Testen gør dette igennem 4 stadier, direkte oversat til: arranger, handling, kontrol og rengøring. Det vigtigste i denne form for test er ikke hvordan eller hvorfor visse ting sker, men hvad specifikt der sker.(Pytest, u.d)

Arranger/forbered er her hvor vi gør alt klar til den egentlige test, stiller alt klar, så ager stadiet kan gøre sin ting uden komplikationer, Det kan betyde, at man opretter objekter, indsætter testdata i en database, starter eller stopper services. Pytest, som er det bibliotek jeg vil bruge, beskriver det som at sætte dominobrikker op (Pytest, u.d).

handlingstadiet er her hvor testen faktisk sker, ofte i form af funktionskald. Det virker ved at ændre systemets tilstand for at tillade tests (Pytest, u.d).

kontrol, Her ser man på systemets nye tilstand og vurderer, om den matcher det forventede. Det er her, man afgør, om testen består eller fejler. Hvis man for eksempel forventer, at en værdi er blevet sat til "green", tjekker man præcist dette. Det er i dette trin, man indsamler bevis for om systemets adfærd stemmer overens med forventningerne.

Rengøring er her hvor testen nulstiller og sørger for at testen ikke kommer til at have nogen effekt på det egentlige program, eller andre tests der kører på samme tid.

Testresultater

Vi ser ud fra vores testresultater at de funktioner vi kan teste, fungerer som forventet og uden komplikationer, ud fra dette kan vi se at vores applikation opfylder halvdelen af de test som vi vil have den til at klare(Figur 5).

Grunden til at halvdelen af testene bliver skippet er, at den ene attribut ikke godtages og ikke kan vises visuelt, dette sker da tkinter ikke kan testes da testene ikke kan starte applikationen for at teste den, dermed bliver de bare skippede da de ikke kan testes. Det vigtigste vi ser fra disse test, er at ingen af dem fejler, derfor opnår vores test det de skulle, nemlig at teste at det ikke var komplikationer i vores funktioner, og de der blev testet, havde ingen. Dermed succes.

Konklusion

I dette projekt har vi udviklet en Python applikation i form af en initiative tracker til Dungeons and Dragons med fokus på struktureret programmering og brug af trelagsmodellen. Applikationen løser et konkret problem som ofte opstår under spillet nemlig at holde styr på turrækkefølgen mellem spillere og fjender på en overskuelig måde. Ved at digitalisere denne proces har vi skabt et værktøj som gør det lettere for gamemasteren at administrere spillets gang uden at miste overblik.

Projektet lever op til de opstillede krav ved at være relativt let at bruge også for nybegyndere og ved at have en klar og intuitiv brugergrænseflade. Opdelingen mellem data logik og præsentation har gjort koden mere overskuelig og nemmere at arbejde videre med hvilket har været en stor fordel under udviklingen. Samtidig har brugen af en database sikret at data håndteres struktureret og konsekvent på tværs af applikationen.

Arbejdsprocessen har givet en bedre forståelse for samspillet mellem de forskellige lag i et program og for vigtigheden af god struktur og planlægning i større projekter. Gruppefordelingen har gjort det muligt at fordybe sig i forskellige dele af projektet samtidig med at logiklaget krævede samarbejde og fælles forståelse. Brugen af AI som støtteværktøj har bidraget positivt til processen uden at overtage udviklingen og har fungeret som en hjælp til problemløsning og inspiration.

Testarbejdet har vist at de dele af programmet som kunne testes fungerede som forventet og uden fejl. Begrænsningerne ved test af GUI med tkinter har gjort at ikke alle funktioner kunne testes automatisk men de gennemførte tests giver stadig en god indikation af programmets stabilitet.

Samlet set har projektet opfyldt formålet med opgaven og demonstrerer en funktionel Python applikation bygget med trelagsmodellen. Projektet har styrket både de tekniske færdigheder inden for programmering og forståelsen for programstruktur design og samarbejde.

Bibliografi

Pytest. (u.d). *Anatomy of a test*. Hentet 25. 3 2026 fra [pytest.org](https://docs.pytest.org/en/stable/explanation/anatomy.html#anatomy-of-a-test):
<https://docs.pytest.org/en/stable/explanation/anatomy.html#anatomy-of-a-test>

Bilag

D&D Initiative Tracker

Add Character

Name:

Initiative:

Type:

hero

Image:

No image selected

Select Image

Add Character

← Previous Turn

No characters yet

Next Turn →

Turn Order (Highest Initiative First)

Select a character to view image

Remove Selected

Clear All

Figur 4, 5. iteration af applikationen

```
python -m pytest tests/guitest.py -v
===== test session starts =====
platform win32 -- Python 3.12.10, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\Stefan\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Pyth
on.3.12_qbz5n2kfra8p0\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Stefan\OneDrive\Dokumenter\python\Eksamensprojekt\GitForEksamenOpgave
collected 10 items

tests/guitest.py::TestHelperFunctions::test_get_color_for_hero PASSED [ 10%]
tests/guitest.py::TestHelperFunctions::test_get_color_for_ally PASSED [ 20%]
tests/guitest.py::TestHelperFunctions::test_get_color_for_enemy PASSED [ 30%]
tests/guitest.py::TestHelperFunctions::test_get_color_for_unknown_type PASSED [ 40%]
tests/guitest.py::TestHelperFunctions::test_get_color_for_empty_string PASSED [ 50%]
tests/guitest.py::TestMainWindow::test_main_window_initialization SKIPPED (GUI test skipped due to display issues: Mock object has no...) [ 60%]
tests/guitest.py::TestMainWindow::test_update_character_list_empty SKIPPED (GUI test skipped: Mock object has no attribute 'tk') [ 70%]
tests/guitest.py::TestMainWindow::test_update_character_list_with_characters SKIPPED (GUI test skipped: Mock object has no attribute ...) [ 80%]
tests/guitest.py::TestImageHandling::test_on_select_image_no_file_selected SKIPPED (GUI test skipped: Mock object has no attribute 'tk') [ 90%]
tests/guitest.py::TestImageHandling::test_on_select_image_valid_file SKIPPED (GUI test skipped: Mock object has no attribute 'tk') [100%]

===== 5 passed, 5 skipped in 0.11s =====
```

Figur 5, pytest unittest for GUI-delen af applikationen